STACK TECHNIQUE — JULIENBOURCET.FR

De la source .tsx au navigateur — comment chaque outil s'imbrique

01 · SOURCE

Fichiers .tsx — React + TypeScript

Le code source que tu écris. Chaque composant est un fichier **.tsx** — du JSX (syntaxe HTML dans JS) typé avec TypeScript. TypeScript ajoute les types : ça détecte les erreurs avant que le navigateur ne les voit.

```
src/
├── main.tsx ← point d'entrée, monte React dans le DOM
├── app/
│   ├── App.tsx ← racine de l'app, branche le Router
│   ├── routes.tsx ← définit les 4 URLs (/ /cases /about /contact)
│   ├── pages/
│   │   ├── Homepage.tsx
│   │   ├── CasesPage.tsx
│   │   ├── AboutPage.tsx
│   │   └── ContactPage.tsx
│   └── components/ ← composants réutilisables (nav, boutons…)
└── styles/index.css ← Tailwind + focus-visible
```

React 18 · TypeScript · JSX → TSX

transpile + bundle

02 · OUTIL DE BUILD

Vite — le compilateur

Vite lit tous tes fichiers .tsx, les transforme en JavaScript pur que le navigateur comprend, et regroupe tout en quelques fichiers optimisés. Il gère aussi le serveur de dev local avec hot reload instantané.

1. Lit main.tsx →
2. Compile TypeScript → JS →
3. Compile Tailwind → CSS →
4. Tree-shaking (supprime le code inutilisé) →
5. Minification + hash des noms de fichiers

vite build · @vitejs/plugin-react · @tailwindcss/vite · ESM natif

génère

03 · BUILD OUTPUT

/dist — fichiers statiques

Le résultat de `npm run build`. Des fichiers statiques : pas de serveur Node, pas de base de données. N'importe quel hébergeur peut les servir.

```
dist/
├── index.html              ← unique HTML, point d'entrée
├── assets/index-[hash].js  ← tout le JS bundlé + minifié
└── assets/index-[hash].css ← tout le CSS (Tailwind purgé)
```

upload FTP

04 · HÉBERGEMENT

Serveur — public_html/

Les fichiers /dist sont uploadés via FTP dans le dossier **public_html** de l'hébergeur. Le serveur ne fait que les servir — il n'exécute rien. C'est un site statique.

FTP upload | Fichiers statiques | Pas de Node en prod

HTTP request

05 · NAVIGATEUR — RUNTIME

React s'exécute côté client

Le navigateur télécharge index.html → charge le JS → React monte l'application dans la `<div id="root">`. À partir de là, toute la navigation est gérée par JavaScript — sans rechargement de page.

React Router
Écoute l'URL. Affiche la bonne page sans rechargement.

Framer Motion
Anime les composants. Transitions de page, springs, stagger.

Tailwind CSS
Classes utilitaires dans le HTML généré par React.

Virtual DOM
React ne redessine que ce qui change — pas toute la page.

LIBRAIRIES CLÉS

React
Construit l'interface en composants. Gère le state et le re-render automatique quand les données changent.

TypeScript
Ajoute les types au JavaScript. Détecte les erreurs à l'écriture, pas à l'exécution.

Vite
Compile et bundle le code source. Dev server ultra-rapide avec HMR (hot module replacement).

React Router
Gère les URLs (/cases, /about…) côté client. La page ne se recharge pas lors de la navigation.

Framer Motion
Animations déclaratives dans les composants React. Springs, stagger, transitions de page.

Tailwind CSS
Classes CSS utilitaires écrites directement dans le JSX. Vite purge les classes inutilisées au build.

● Code source ● Compilation (dev machine) ● Fichiers produits ● Serveur ● Navigateur (runtime)